URL Shortener API

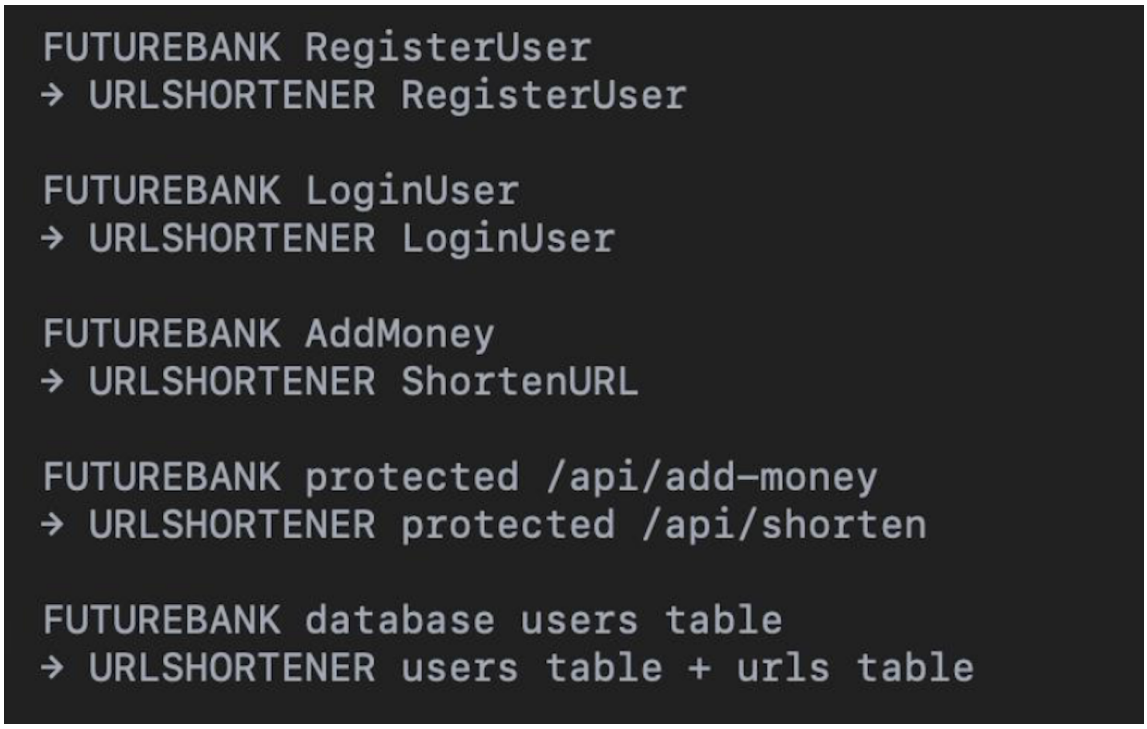
README and Testing Evidence

These are my notes explaining what I built, how the project works, and how I tested the main API flow using Postman and pgAdmin.

Area	What I did
Project goal	Built a Go API that turns a long URL into a short URL.
Main features	Register, login, shorten URL, redirect, delete URL, logout.
Database	PostgreSQL with users and urls tables.
Security	Passwords are hashed with bcrypt and protected routes use JWT tokens.
Testing tools	Postman for API requests and pgAdmin for database checks.

What I reused from FUTUREBANK

I used the same layered pattern from FUTUREBANK. RegisterUser and LoginUser stayed very similar. The new researched part was ShortenURL: generating a short code, saving it with the original URL, redirecting using the code, and deleting it.



Project Structure

I organised the code into packages so each part has a clear job.

```
urlShortener/
■■■ handlers/ -> receives Postman/HTTP requests
■■■ middleware/ -> checks JWT tokens
■■■ models/ -> defines users and urls tables
■■■ repository/ -> talks to PostgreSQL using GORM
■■■ routes/ -> connects endpoints to handlers
■■■ service/ -> main business logic
■■■ utils/ -> helper functions such as password hashing and short code generation
■■■ main.go -> starts the app
■■■ README.md -> explains the project
```

Main Endpoints

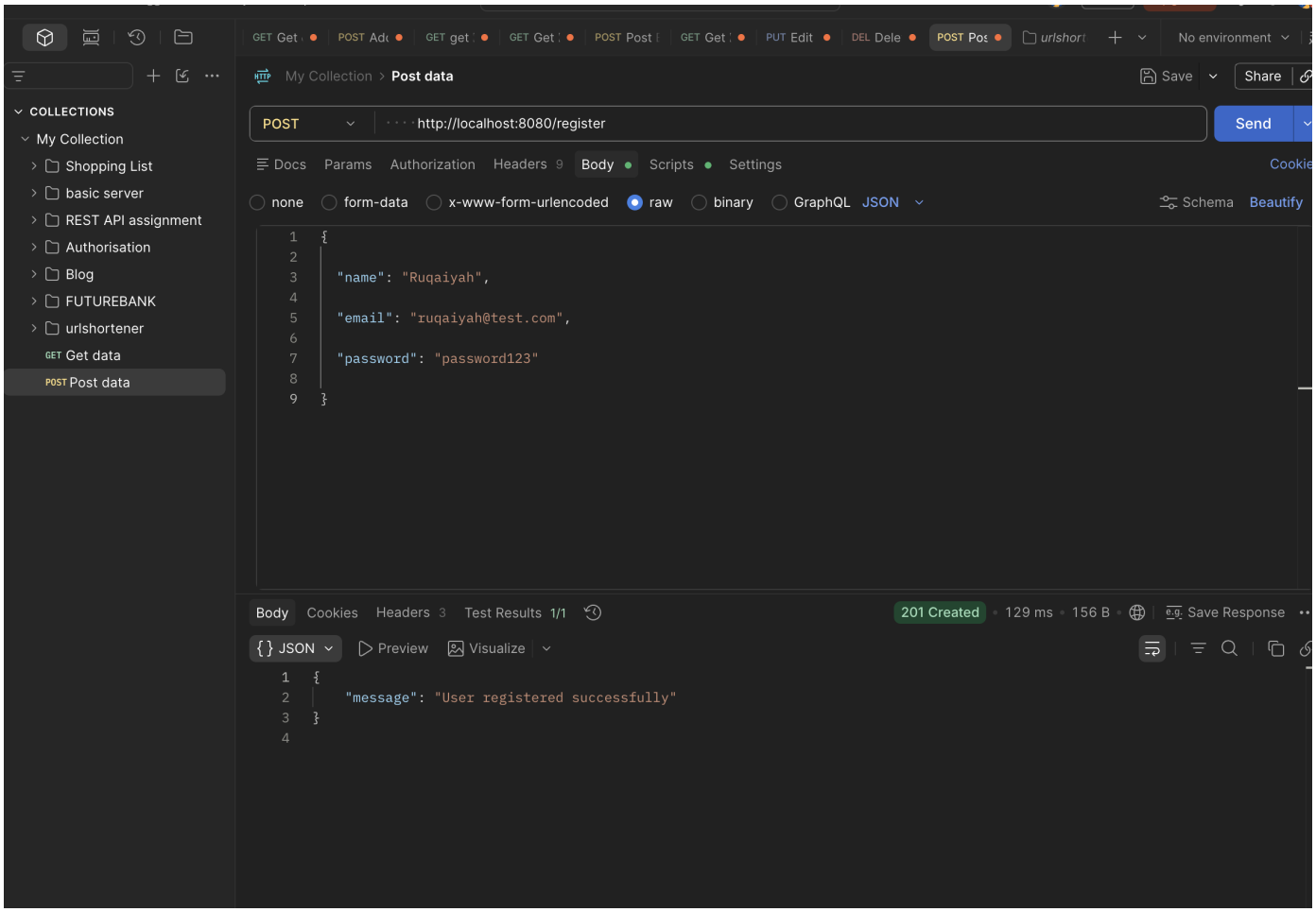
Method	Endpoint	Purpose
POST	/register	Create a new user
POST	/login	Login and return a JWT token
POST	/api/shorten	Shorten a URL - protected route
GET	/{code}	Redirect short URL to original URL
DELETE	/api/url/{code}	Delete a short URL - protected route
POST	/logout	Return logout message

How to run it locally

```
1. Create PostgreSQL database: urlshortener
2. Add .env values:
DB_HOST=localhost
DB_USER=postgres
DB_PASSWORD=your_password
DB_PORT=5432
DB_NAME=urlshortener
DB_SSLMODE=disable
JWT_SECRET=your_secret_key
BASE_URL=http://localhost:8080
3. Run:
go mod tidy
go run main.go
```

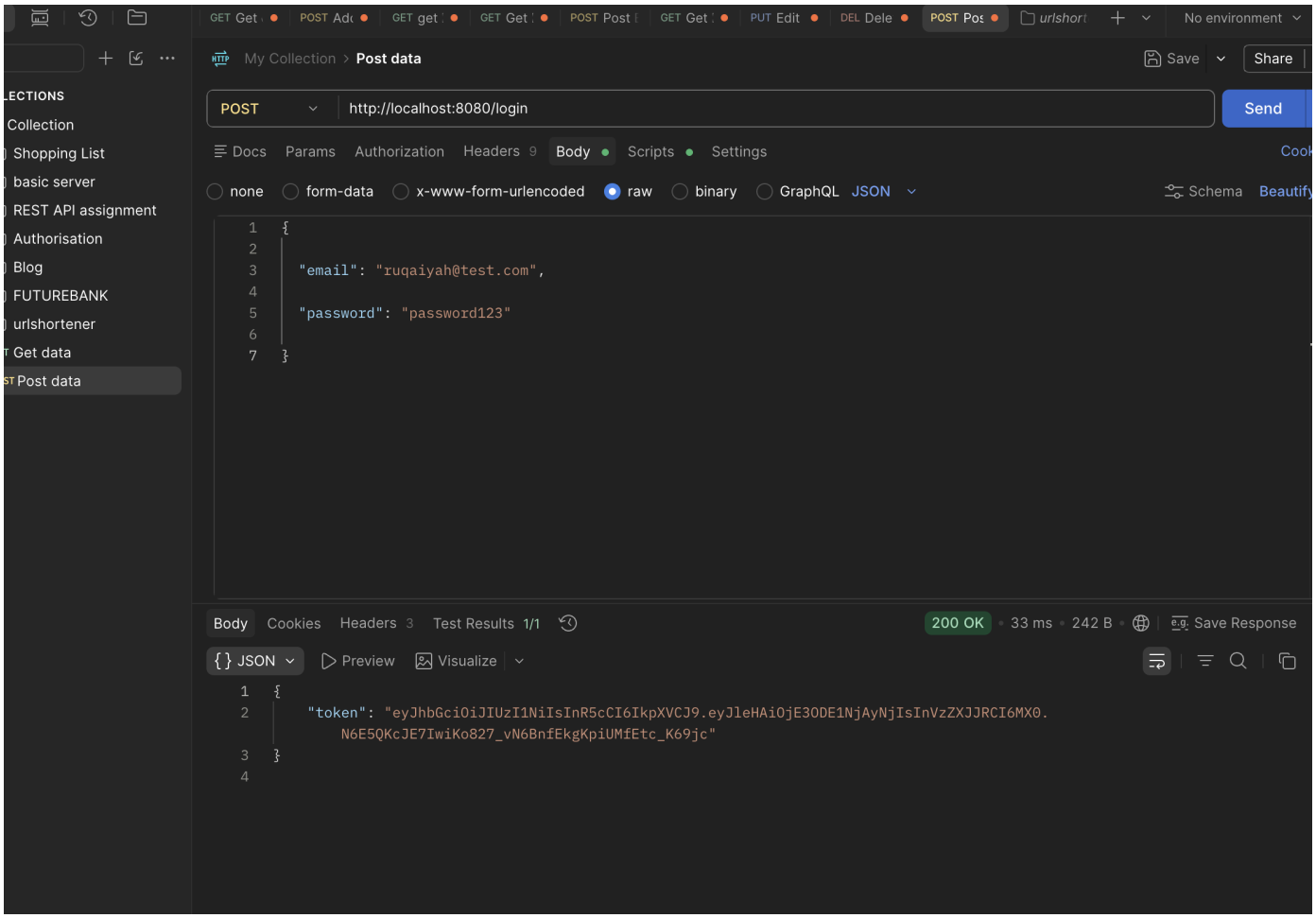
1. Register user

I tested POST /register in Postman. I sent name, email and password as JSON. The API returned 201 Created with a success message, so the user was created.



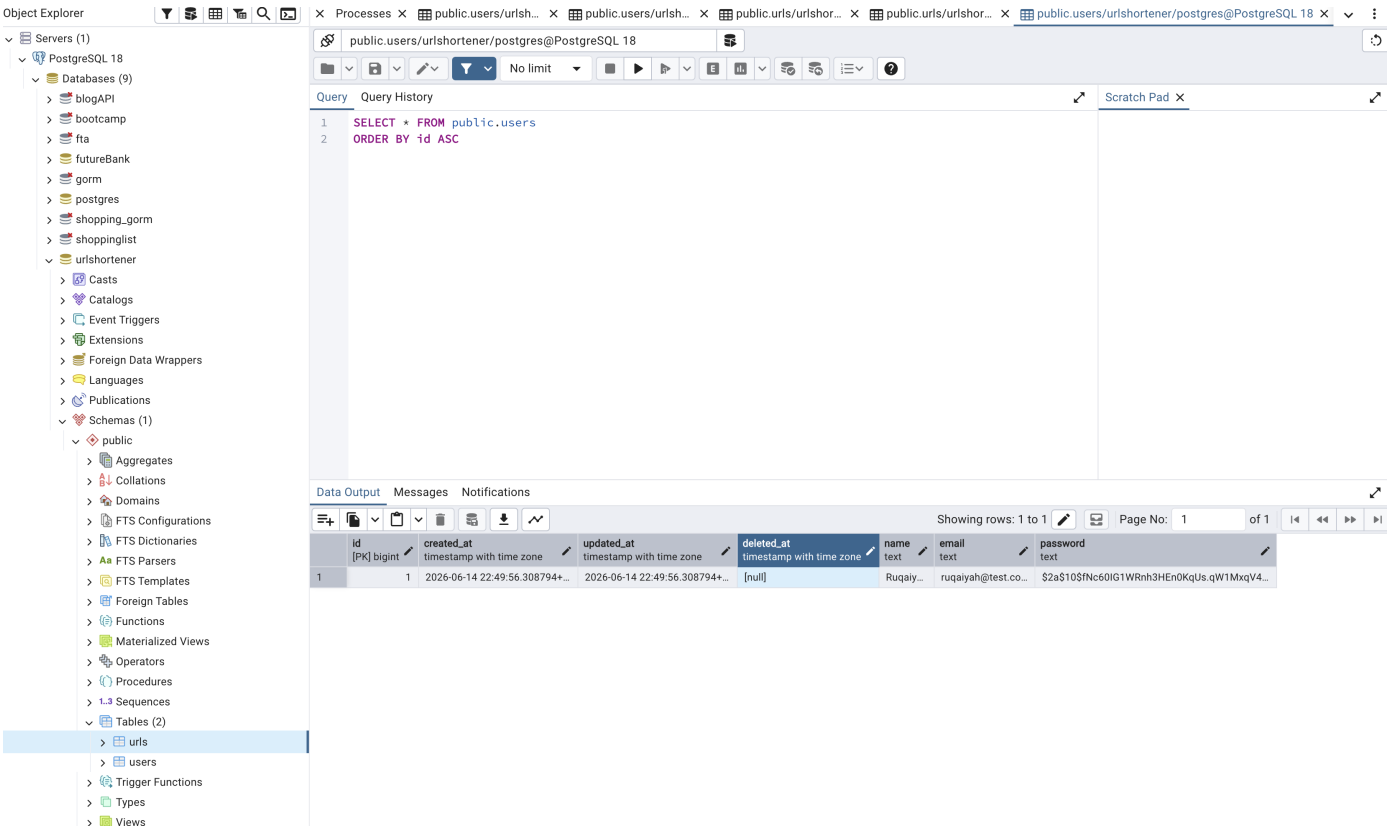
2. Login user

I tested POST /login with the same email and password. The API returned a JWT token. I copied this token to use in protected routes.



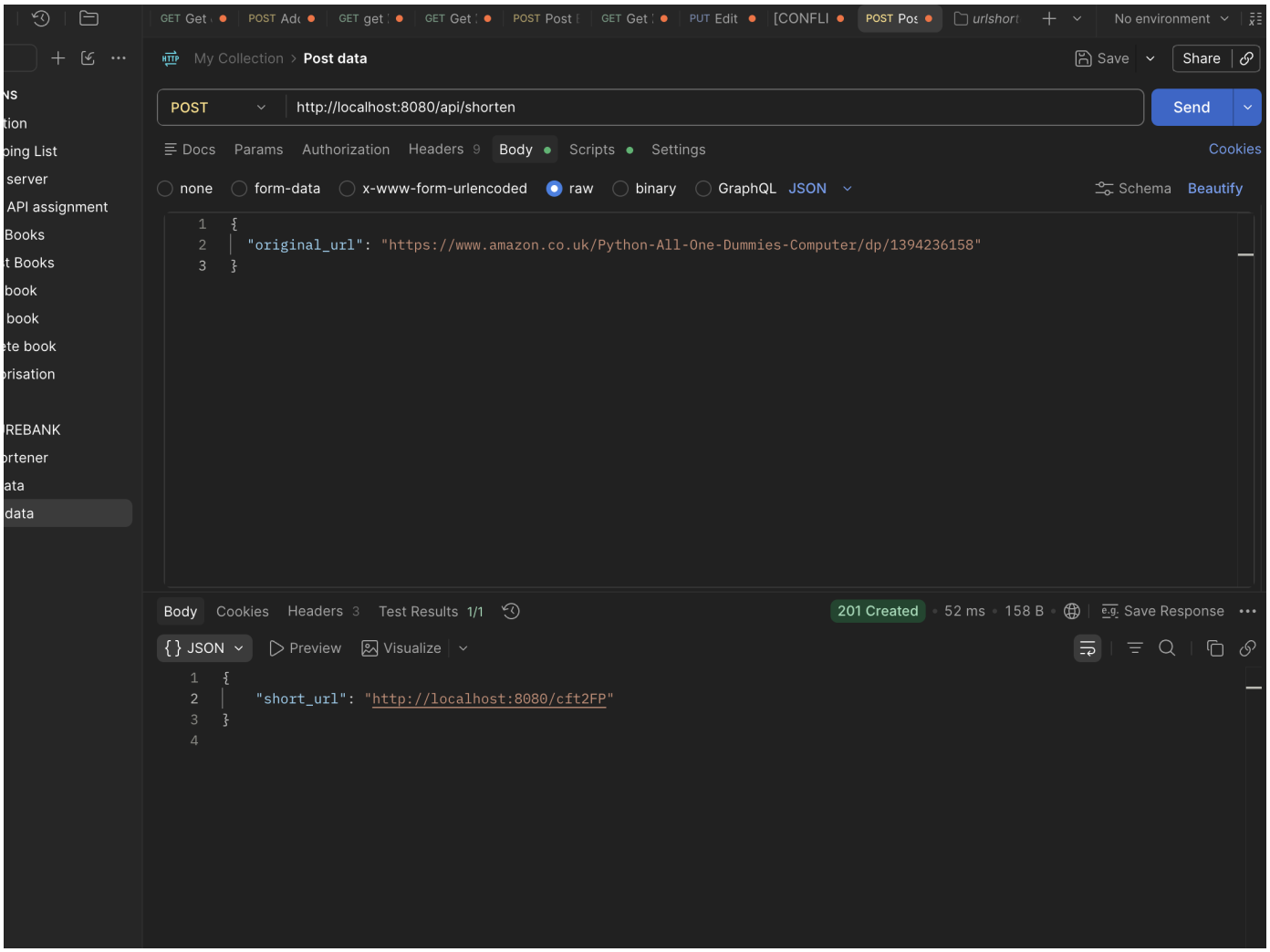
3. Check users table in pgAdmin

I checked the users table in pgAdmin. The user was saved and the password was stored as a bcrypt hash, not plain text.



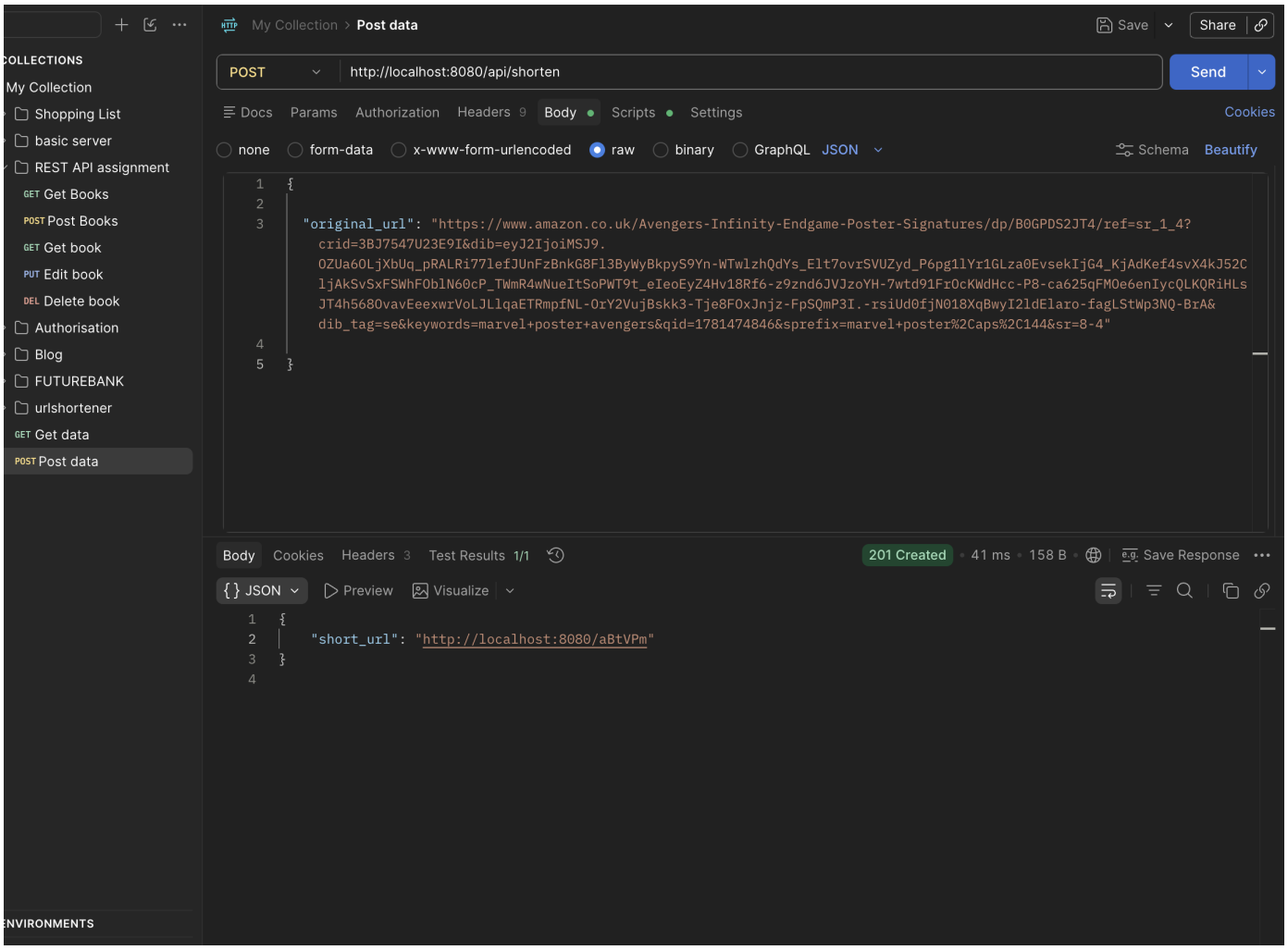
4. Shorten a URL

I tested POST /api/shorten. I added the Authorization header with Bearer token, sent an original URL, and received a localhost short URL.



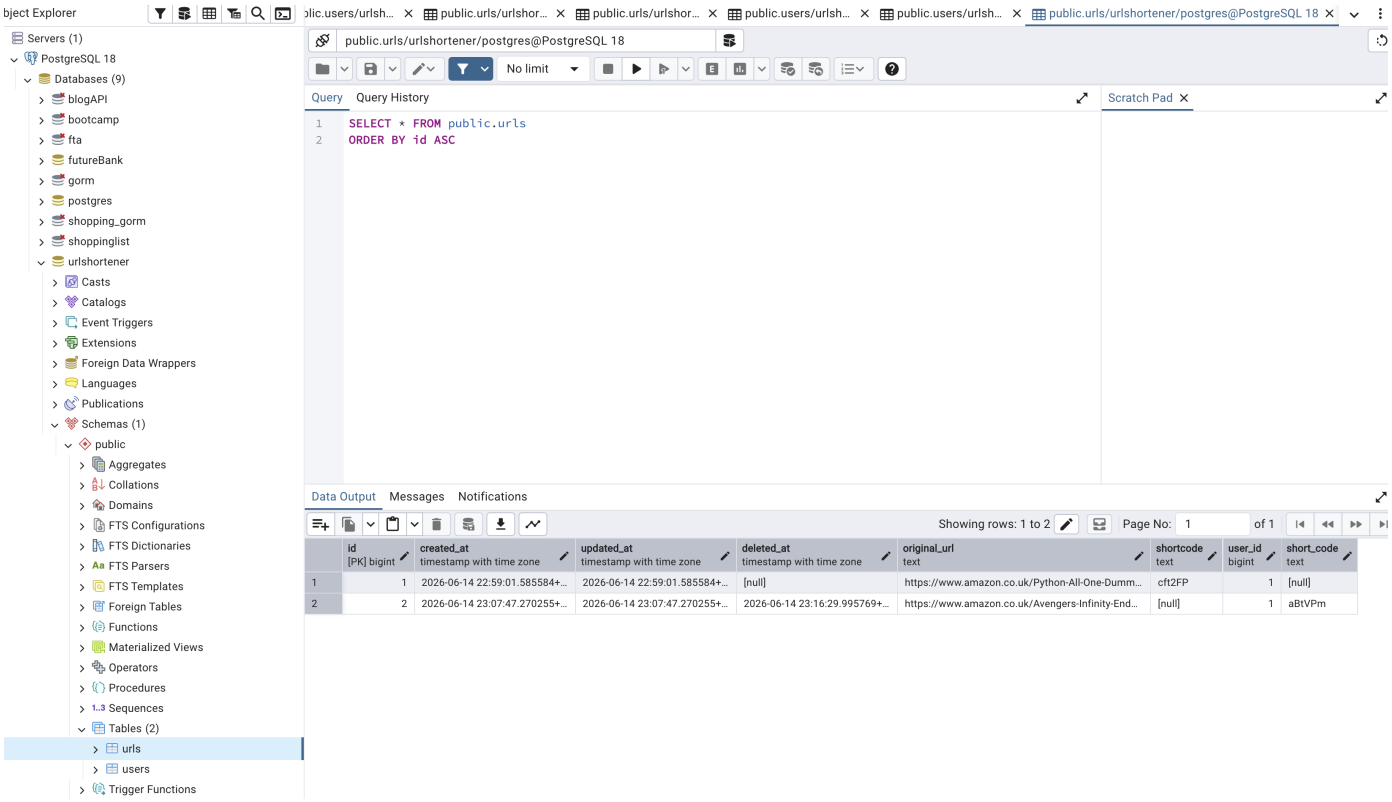
5. Shorten another URL

I repeated the shorten request with another long URL. This showed the app could generate a new short code and save another record.



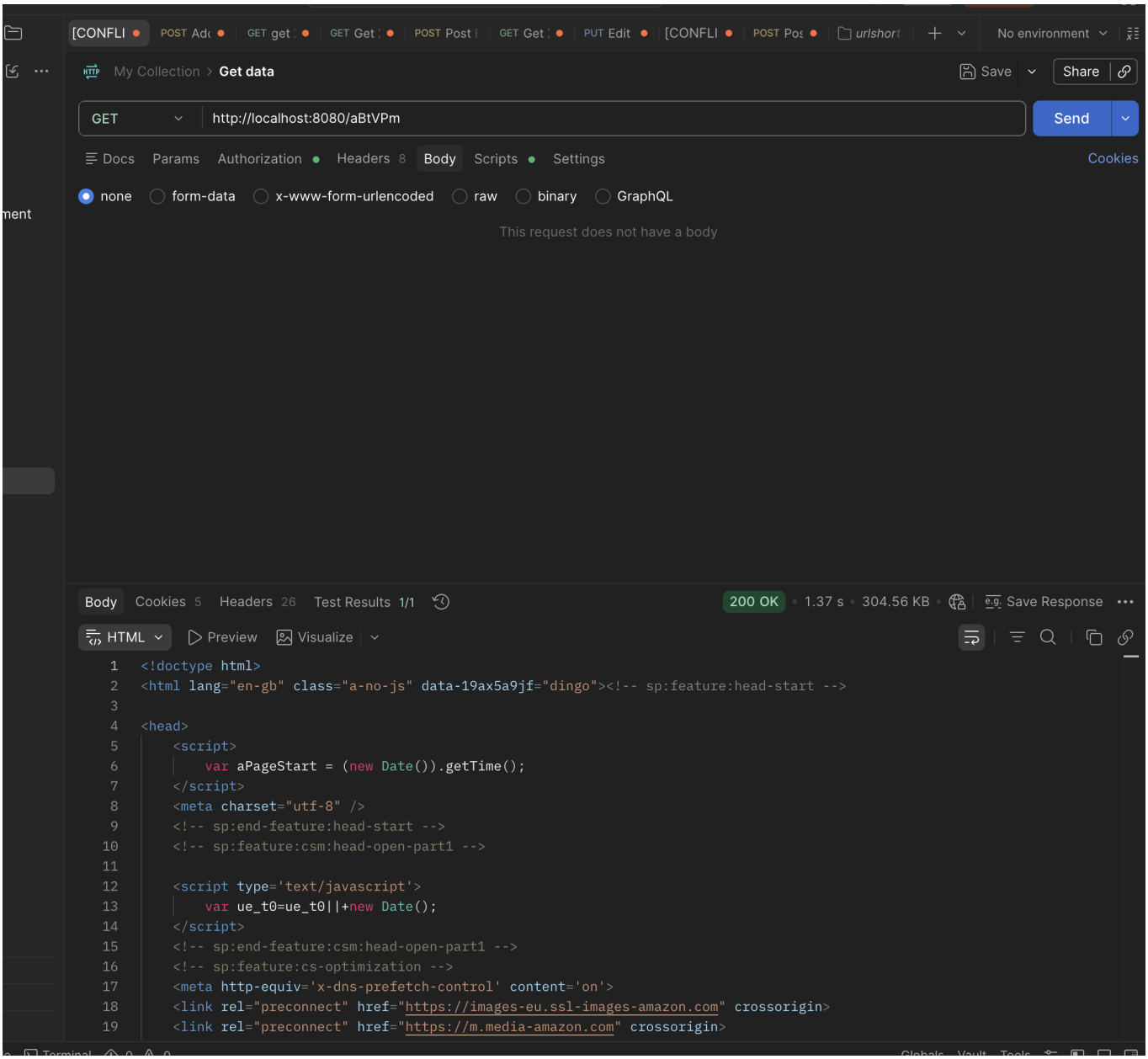
6. Check urls table in pgAdmin

I checked the urls table. I could see the original URL, short code and user_id, which proved the short links were saved in the database.



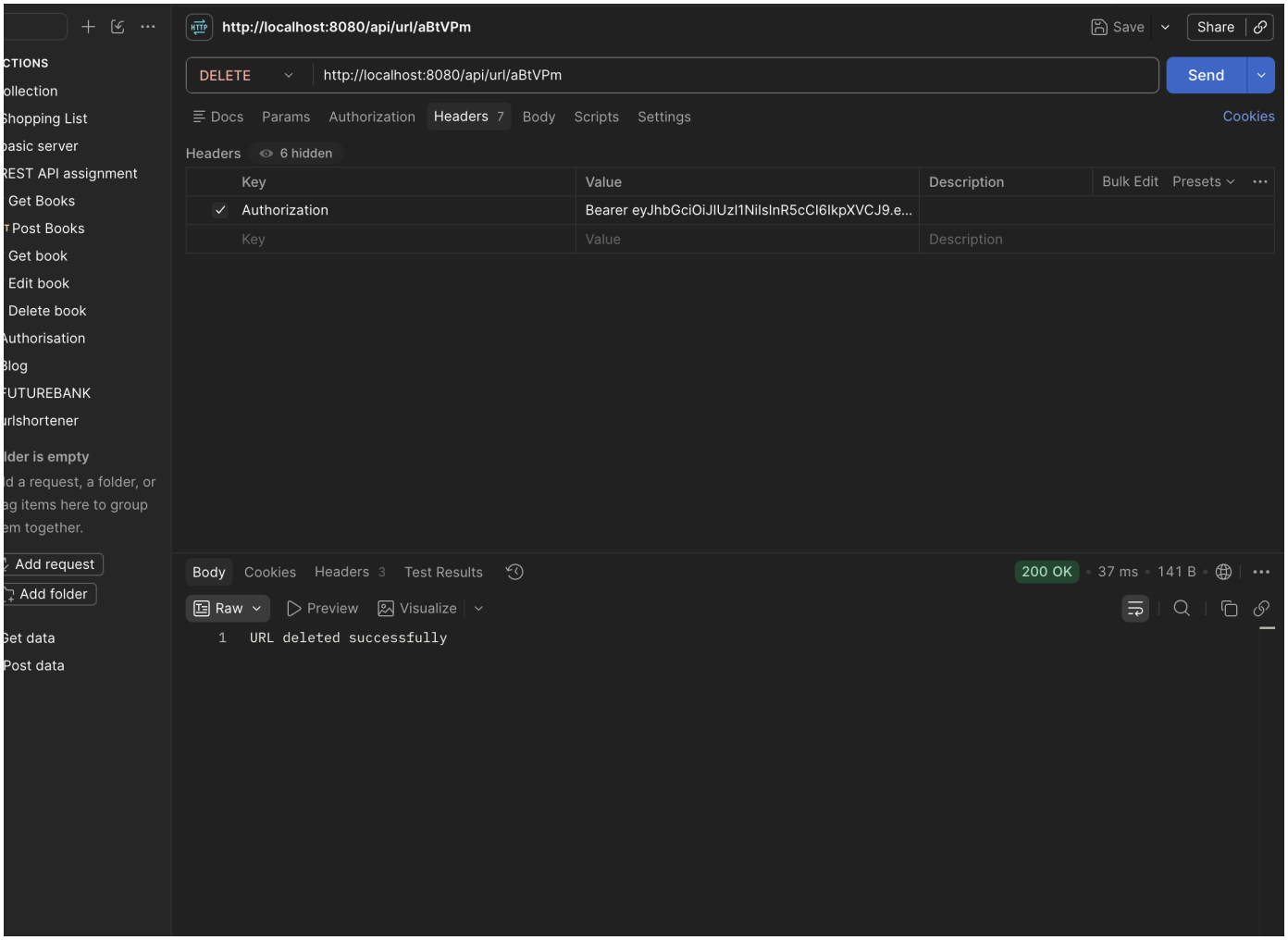
7. Redirect using short URL

I tested GET /{code}. Postman followed the redirect and showed the HTML response from Amazon. This proved the short code redirected to the original URL.



8. Delete short URL

I tested DELETE /api/url/{code} with the Bearer token. The API returned URL deleted successfully.



Testing order I followed

```
1. POST /register
2. POST /login
3. Copy JWT token
4. POST /api/shorten with Authorization: Bearer token
5. GET /{code} to test redirect
6. DELETE /api/url/{code} with Authorization: Bearer token
7. GET /{code} again to confirm it was deleted
```

Final reflection

By the end, the full journey worked: I registered a user, logged in, used the token to shorten a URL, checked the database, redirected using the short code, deleted the URL, and confirmed it could no longer be found.

This helped me understand how the FUTUREBANK structure can be reused for a different project. The handlers deal with Postman requests, the service layer contains the logic, the repository talks to the database, and middleware protects routes using the JWT token.